

Finite Automata in Haskell

V. Velkey, W. Vromen

June 1, 2023

Overview

- ▶ Theory overview: DFA-s, NFA-s, ϵ -NFA-s, Regular expressions
- ▶ DFA -s: implementation, evaluation of words
- ▶ NFA-s and ϵ -NFA-s: implementation, evaluation
- ▶ Powerset construction: ϵ -NFA to DFA
- ▶ RegExp: implementation, word generation
- ▶ DFA to RegExp
- ▶ RegExp to ϵ -NF
- ▶ Tests

DFA -s: implementation, evaluation of words

Design choice: words with unknown symbols will be *rejected*.

```
type Symbol = Char
data DFA a = DFA { states :: [a]
    , alphabet :: [Symbol]
    , delta :: Symbol -> a -> a
    , start :: a
    , acceptstate :: [a]}

evaluate :: (Eq a, Ord a) => DFA a -> String -> Bool
evaluate df st = all ('elem' alphabet df) st && -- all characters in
    string are in the alphabet
    stateArr (start df) st 'elem' acceptstate df where
        stateArr q [] = q
        stateArr q (x:xs) = stateArr (delta df x q) xs
```

NFA implementation

Transitioning between lists of states: double recursion.

```
data NFA a = NFA { statesNF :: [a]
                  , alphabetNF :: [Symbol]
                  , deltaNF :: Symbol -> a -> [a]
                  , startNF :: a
                  , acceptstateNF :: [a]}

evaluateNF :: Eq a => NFA a -> String -> Bool
evaluateNF nf st = all ('elem' alphabetNF nf) st && -- captures elements
  of string in alphabet
    any ('elem' acceptstateNF nf) (stateArrNF' [startNF nf] st)
      where
        stateArrNF' qs [] = qs
        stateArrNF' [] _ = []
        stateArrNF' [q] (x:xs) = stateArrNF' (deltaNF nf x q)
          xs --recursion on string
        stateArrNF' (q:qs) (x:xs) = stateArrNF' [q] (x : xs) '
          union' stateArrNF' qs (x : xs) --recursion on
          statespace
```

RegExp implementation: data type

```
data RegExp = Empty | Epsilon | R [Symbol] | Union RegExp RegExp
            | Star RegExp | Con RegExp RegExp | Plus RegExp
  deriving (Show, Eq)
```

Design choices:

- ▶ not split empty from non-empty expressions
- ▶ Epsilon versus R []
- ▶ R [Symbol] versus R Symbol

For convenience:

```
regExpUnion :: [RegExp] -> RegExp
regExpUnion = simplify . foldr Union Empty
```